Python 期中考規模練習題

物件導向程式設計（OOP）— 完整作答程式碼、逐步解析與執行驗證手冊

建議時間：90 分鐘 | 試卷總分：100 分 | 使用環境：Python 3.10+ / 內建函式庫 | 檔案目錄：物件導向期中

第一題目：讀程式、追蹤變化（總分 20 分）

請依 Python 的實際執行順序作答，並寫出每一步的值；不要只憑直覺猜測。

1A. 數值與型別 6 分

題目原始程式碼

```
a = round(42.5)
b = round(43.5)
c = 7 / 2

print(a)
print(b)
print(c)
print(type(c))
```

作答內容與逐步解析：

1. **寫出 a、b、c 的值：**
   - `a = 42`
   - `b = 44`
   - `c = 3.5`
2. **寫出四行輸出的內容：**

```
42
44
3.5
<class 'float'>
```

3. **說明為什麼 a 與 b 的結果不同：**
   Python 的內建 `round()` 函式採用「**銀行家捨入法（Round half to even，取至最接近的偶數）**」規則。
   當小數剛好位於兩個整數正中間（.5）時：
   - `42.5` 介於 42 與 43 之間，最接近的偶數是 **42**。
   - `43.5` 介於 43 與 44 之間，最接近的偶數是 **44**。

   因為兩者剛好相鄰的偶數不同，故分別捨入為 42 與 44。

1B. 追蹤遞迴呼叫 6 分

題目原始程式碼

```
def gcd(a, b):
    if b == 0:
        return a
    return gcd(b, a % b)
```

作答內容與逐步解析：

1. **gcd(55, 21) 每次呼叫的參數追蹤表：**

呼叫順序	參數 a	參數 b	計算步驟 / 備註
1	55	21	55 % 21 = 13
2	21	13	21 % 13 = 8
3	13	8	13 % 8 = 5
4	8	5	8 % 5 = 3
5	5	3	5 % 3 = 2
6	3	2	3 % 2 = 1
7	2	1	2 % 1 = 0
8	1	0	滿足停止條件 b == 0

2. **遞迴停止時符合的條件：** `b == 0`（第 8 次呼叫時成立）。

3. **最後回傳值：** `1`。

1C. 串列別名 4 分

題目原始程式碼

```
left = [2, 4]
right = left
right.append(6)
left[0] = 10
print(left)
print(right)
```

作答內容與逐步解析：

1. **寫出兩行輸出：**

```
[10, 4, 6]
[10, 4, 6]
```

2. **用一句話說明 left 與 right 的關係：**
   `left` 與 `right` 參照（指向）記憶體中同一個串列物件（`right` 是 `left` 的別名），因此透過任一名稱修改串列，另一個變數所見的內容也會同步改變。

1D. 字串索引與切片 4 分

題目原始程式碼

```
word = "Python"
print(word[0])
print(word[1:4])
print(len(word))
```

作答內容與逐步解析：

1. **寫出三行輸出：**

```
P
yth
6
```

2. **指出切片的結尾索引是否包含在結果中：**
   不包含。Python 的切片語法 `[start:stop]` 為「左閉右開」區間，包含起始索引，但不包含結尾索引。因此 `word[1:4]` 只會擷取索引 1, 2, 3 的字元（'y', 't', 'h'），不包含索引 4 的字元（'o'）。

第二題：成績清單摘要函式（總分 30 分）

題目需求： 請完成函式 `print_score_summary(scores, passing_score)`。函式接收一個整數串列與及格門檻，依指定格式輸出摘要。

完整解答原始碼 (Q2_SCORE_SUMMARY.PY)

```
def print_score_summary(scores, passing_score):
    """
    計算並依序印出成績清單的摘要資訊。

    參數:
        scores (list): 整數分數串列（保證至少有一個分數）
        passing_score (int): 及格門檻分數
    """
    # 步驟 1 & 2: 初始化總分與及格人數
    total_score = 0
    passing_count = 0

    # 步驟 3 & 4: 走訪 scores 累加總分，並判斷是否達到及格門檻
    for score in scores:
        total_score += score
        if score >= passing_score:
            passing_count += 1

    # 步驟 5: 計算平均分數（直接除法，不需自行四捨五入）
    count = len(scores)
    average = total_score / count

    # 步驟 6: 使用 f-string 依序輸出四行摘要
    print(f"筆數：{count}")
    print(f"總分：{total_score}")
    print(f"平均：{average}")
    print(f"及格人數：{passing_count}")
```

測試資料與驗證輸出：

測試 1（題目範例）：

```
scores = [72, 90, 64, 88, 96]
print_score_summary(scores, 70)
```

```
筆數：5
總分：410
平均：82.0
及格人數：4
```

測試 2（分數等於門檻亦算及格）：

```
print_score_summary([50, 60, 70], 60)
```

```
筆數：3
總分：180
平均：60.0
及格人數：2
```

評分重點自我查檢

- ✓ 初始化與計算總分正確 (8 分)
- ✓ 逐項判斷及格（使用 `>=`）並正確計數 (8 分)
- ✓ 筆數與平均計算正確 (6 分)
- ✓ 輸出標籤（筆數：/總分：/平均：/及格人數：）、順序與格式一致 (5 分)
- ✓ 函式能用不同測試資料重複使用 (3 分)

第三題：找出符合條件的長回文單字（總分 30 分）

題目需求： 判斷單字是否為回文（忽略大小寫，使用 `lower()`、`reversed()` 與 `''.join(...)`）。篩選長度至少為 `min_length` 且是回文的單字，維持原始順序與拼寫回傳。

完整解答原始碼 (Q3_PALINDROME.PY)

```
def is_palindrome(word):
    """
    判斷單字是否為回文（忽略大小寫）。

    步驟:
    1. 使用 lower() 建立不分大小寫的字串。
    2. 以 reversed() 與 ''.join(...) 建立反轉字串。
    3. 比較兩者是否相等，相同回傳 True，否則 False。
    """
    lower_word = word.lower()
    reversed_word = ''.join(reversed(lower_word))
    return lower_word == reversed_word


def find_long_palindromes(words, min_length):
    """
    找出串列中長度至少為 min_length 的所有回文單字。

    步驟:
    4. 建立空串列。
    5. 只有在「長度至少為 min_length」且「是回文」時，加入原始單字。
    6. 回傳結果串列（不在函式內直接印出）。
    """
    result = []
    for word in words:
        if len(word) >= min_length and is_palindrome(word):
            result.append(word)
    return result
```

測試資料與驗證輸出：

```
words = ["Noon", "level", "Python", "Radar", "civic", "data"]

# 單元測試 is_palindrome
print(is_palindrome("Noon"))    # 預期: True
print(is_palindrome("Python"))  # 預期: False

# 呼叫 find_long_palindromes(words, 4)
print(find_long_palindromes(words, 4))
# 輸出: ['Noon', 'level', 'Radar', 'civic']

# 呼叫 find_long_palindromes(words, 5)
```

```
print(find_long_palindromes(words, 5))
# 輸出: ['level', 'Radar', 'civic']
```

```
True
False
['Noon', 'level', 'Radar', 'civic']
['level', 'Radar', 'civic']
```

評分重點自我查檢

- ✓ 正確完成大小寫正規化與 reversed() + join 反轉 (10 分)
- ✓ is_palindrome 對正例與反例判斷皆正確 (6 分)
- ✓ 正確走訪並同時套用長度與回文條件 (8 分)
- ✓ 正確回傳串列，維持原本順序與拼寫（大小寫未變動）(4 分)
- ✓ 完成正反例與長度限制自我測試 (2 分)

第四題：使用遞迴計算最大公因數（總分 20 分）

題目需求： 以輾轉相除法原理自行設計遞迴函式 `gcd(a, b)`，求非負整數的最大公因數（不可使用 `math.gcd()`）。

完整解答原始碼 (Q4_RECURSIVE_GCD.PY)

```
def gcd(a, b):
    """
    使用遞迴計算整數 a 與 b 的最大公因數 (GCD)。

    核心概念：
    1. 基底條件 (Base Case): 當 b == 0 時，最大公因數即為 a，直接 return a。
    2. 遞迴步驟 (Recursive Step): gcd(a, b) = gcd(b, a % b)。
       每次取餘數後交換參數位置，第二個參數嚴格縮小，保證逐步停止。
    """
    # 步驟 1 & 2: 停止條件
    if b == 0:
        return a

    # 步驟 3 & 4: 遞迴縮小問題
    return gcd(b, a % b)
```

測試資料表格與驗證：

測試函式呼叫	預期結果	實際回傳值	測試狀態	遞迴呼叫次數
`gcd(55, 21)`	1	1	**PASS**	8 次
`gcd(24, 18)`	6	6	**PASS**	3 次 (24,18 -> 18,6 -> 6,0)
`gcd(7, 0)`	7	7	**PASS**	1 次 (直接命中停止條件)
`gcd(18, 24)`	6	6	**PASS**	4 次 (a < b 自動對調)

考前複習與作答檢查重點總結

- ✓ **型別運算**：注意 `/` 恆產生浮點數（float），`round()` 遇 .5 取最近之偶數。
- ✓ **串列別名**：`b = a` 僅複製記憶體位址參照，修改其一會影響另一變數。
- ✓ **字串切片**：切片 `[start:stop]` 左閉右開，長度等於 `stop - start`。
- ✓ **回傳 vs 印出**：題目要求「回傳」務必使用 `return`；要求「印出」才使用 `print()`。
- ✓ **遞迴雙要素**：務必檢查「停止條件（Base Case）」以及「參數縮小朝向終止前進」。